

Manual Técnico do Script password_analyzer.py - Sistema Hórus

Versão do Documento: 1.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python 3 + PostgreSQL + Bibliotecas de Análise (Pandas, Matplotlib, Seaborn)

Este manual técnico descreve em detalhes o script `password_analyzer.py`, que atua como o motor analítico para avaliação de senhas no sistema Hórus. O script é responsável por calcular complexidade, verificar vazamentos contra bases como HIBP, gerar gráficos e relatórios executivos, e analisar tendências temporais das senhas armazenadas no banco de dados. Ele integra com PostgreSQL via SQLAlchemy e produz saídas em formato de dicionários para integração com interfaces web ou dashboards.

O documento explica as funcionalidades principais, configurações, funções disponíveis, restrições de uso e mecanismos de análise implementados. Ele é destinado a administradores, desenvolvedores e mantenedores que precisam compreender ou modificar o comportamento da análise de senhas.

1. Introdução e Propósito

O `password_analyzer.py` é o componente dedicado à análise avançada de senhas no Hórus, uma plataforma de Threat Intelligence para processamento e gestão de credenciais vazadas. Ele avalia a força das senhas com base em critérios como comprimento, variedade de caracteres e complexidade, cruza hashes SHA1 com arquivos de senhas vazadas (HIBP), gera visualizações gráficas em base64 e produz relatórios métricos. O script opera de forma independente via linha de comando ou integrado a outros módulos (ex: `app.py` para dashboards web).

A classe principal, `PasswordAnalyzer`, encapsula todas as lógicas, utilizando Pandas para manipulação de dados, Matplotlib/Seaborn para gráficos e SQLAlchemy para queries otimizadas no PostgreSQL. Ele foca em insights de segurança, como distribuição de força e tendências, para auxiliar na detecção de riscos.

1.1 Dependências e Integrações

- **Linguagem:** Python 3.
- **Banco de Dados:** PostgreSQL (via SQLAlchemy), com engine otimizada (pool de conexões, timeout).
- **Bibliotecas:** Pandas e NumPy (manipulação de dados), Matplotlib e Seaborn (gráficos), re (regex para validações), base64 e io (conversão de gráficos), logging (logs de erros/info).
- **Integrações Externas:** Banco de dados repo (tabela credenciais), arquivo HIBP local (/Dados/HIBP/100k_sha1.txt) para verificação de vazamentos.
- **Scripts Relacionados:** Integra com statistics_dashboard.py (dashboards), app.py (exibição web de análises), credenciais_processor.py (dados processados para análise).

2. Funcionalidades Principais

O `password_analyzer.py` fornece ferramentas analíticas para senhas, com foco em métricas de segurança e visualizações. Abaixo, as principais funcionalidades:

- **Análise de Complexidade:** Calcula pontuação baseada em comprimento ($\geq 8/12$), maiúsculas, minúsculas, números e caracteres especiais. Classifica como Fraca/Média/Forte.
- **Verificação de Vazamentos:** Compara hashes SHA1 de senhas no banco com arquivo HIBP local, retornando estatísticas, gráficos de proporção e lista de senhas comprometidas.
- **Geração de Gráficos:** Produz visualizações em base64 (distribuição de comprimento/complexidade, proporção de força, sites com senhas fracas, tendências temporais) usando Matplotlib/Seaborn.
- **Relatórios e Exportações:** Gera relatórios executivos com métricas principais (média de tamanho/complexidade, distribuição de força, sites críticos) e DataFrames para exportação CSV.
- **Análise de Tendências:** Avalia médias de comprimento e totais de senhas por período (ex: últimos 30 dias), calculando tendências diárias.
- **Validação Individual:** Analisa uma senha específica, retornando detalhes como complexidade, comprimento e presença de tipos de caracteres.
- **Tratamento de Dados:** Usa Pandas para queries SQL, limpeza de nulos e conversões numéricas; logs detalhados para erros.
- **Integração com Banco:** Queries otimizadas para agregações (AVG, COUNT, GROUP BY) no PostgreSQL.

3. Configurações

O script define configurações globais e de ambiente via imports de config.py e variáveis na classe:

- **Temporais e Usuário:** `self.current_time` e `self.current_user` hardcoded para timestamps e metadados em relatórios.
- **Estilos Gráficos:** `sns.set_theme (whitegrid, palette='deep')`, `plt.style.use('default')` para consistência visual.
- **Banco de Dados:** Usa `get_sqlalchemy_url('repo')` para `connection_string`; `SQLALCHEMY_CONFIG` para pool (size, timeout).
- **Arquivo HIBP:** Path hardcoded (`/Dados/HIBP/100k_sha1.txt`); lê como set para interseção eficiente.
- **Cores Gráficos:** Lista fixa (ex: `'#3498db'`, `'#2ecc71'`) para paletas.
- **Outras:** `CODIFICACAO='utf-8'` (de config); logger para `'password_analyzer'`.

Restrições: Path HIBP fixo (mova para config se necessário); sem suporte a outros bancos (apenas PostgreSQL); queries sem schema explícito (assume default).

4. Funções e Métodos Principais

O script define uma classe `PasswordAnalyzer` com métodos modulares. Cada método é descrito abaixo, incluindo parâmetros, funcionalidades e restrições:

- **init(db_config=None)**: Inicializa engine, estilos gráficos, time/user. Raise em erros de conexão.
- **_create_engine()**: Cria engine SQLAlchemy com logging. Restrição: Depende de config.py.
- **analisar_senhas_vazadas()**: Compara hashes com HIBP, calcula percentuais, gera gráfico pie em base64. Retorna dict com gráficos, stats e lista comprometidas. Restrição: Assume coluna 'hash_da_senha'; retorna vazio se arquivo HIBP não encontrado.
- **_gerar_graficos(df)**: Gera múltiplos gráficos (distribuição comprimento/complexidade, proporção força, sites senhas fracas, tendências). Usa histplot/barplot/lineplot. Retorna dict de base64. Restrição: Limpa nulos; bins=20 fixo.
- **analisar_senhas()**: Query principal para stats (totais, médias, distribuições, sites fracos). Chama _gerar_graficos. Retorna dict com stats e gráficos.
- **exportar_relatorio_executivo()**: Gera DataFrame com métricas por site (totais, médias comprimento, min/max). Adiciona metadados. Restrição: Limite 1000 sites.
- **analisar_tendencias(periodo_dias=30)**: Query agregada por data, calcula médias e tendências. Retorna dict com stats temporais. Restrição: Usa INTERVAL PostgreSQL; NaN se vazio.
- **validar_senha(senha)**: Analisa senha individual (complexidade, comprimento, flags caracteres). Retorna dict de resultados. Restrição: Requer string não vazia.
- **calcular_complexidade_senha(senha)**: Pontuação baseada em critérios. Retorna int (0-6).
- **if name == "main"**: Teste de inicialização e chamadas de análise. Print stats básicas.

5. Restrições e Segurança

- **Dados de Entrada:** Senhas do banco assumidas não nulas/válidas; validações regex simples (sem padrões avançados como repetições).
- **HIBP:** Verificação local (sem API externa); hashes lowercase para case-insensitive. Restrição: Arquivo deve existir; limitado a 100k hashes.
- **Performance:** Queries agregadas eficientes, mas para grandes datasets (>1M rows), ajuste pool_size. Pandas em memória (risco OOM em volumes extremos).
- **Segurança:** Hashes SHA1 usados (não armazena senhas claras); logs sem dados sensíveis. Raise em erros para propagação.
- **Gráficos:** Base64 para web; tamanhos fixos (ex: 12x6). Restrição: Não responsivos por default.
- **Erros:** Tratados com logger e raise; retorna estruturas vazias em falhas (ex: DF vazio).
- **Restrições Gerais:** Foco em análise estática (não real-time); sem suporte a outros formatos de vazados (apenas SHA1 txt). Queries PostgreSQL-specific (DATE, INTERVAL).

Nota Final

O `password_analyzer.py` é focado em insights de segurança, mas alterações devem preservar precisão das métricas e conformidade (ex: LGPD para senhas sensíveis).

Consulte o código para extensões ou integre com ML para análises mais avançadas (ex: entropia). Em produção, atualize HIBP regularmente e monitore logs para erros de conexão.